

RC & RA

CPE241 Database System

Example Domain: Medical

Patient(pID, pName, pAge)

Doctor(dID, dName, dAge)

Appointment(pID, dID, aDatetime)

Basic Queries

Select patients with name = 'Adam'

RA

$\sigma_{pName='Adam'}(Patient)$

TRC

$\{p \mid p \in Patient \wedge p.pName = 'Adam'\}$

DRC

$\{pID, pName, pAge \mid Patient(pID, pName, pAge) \wedge pName = 'Adam'\}$

SQL

SELECT *

FROM Patient

WHERE pName = 'Adam';

Add another condition

“Find patient whose name is ‘Adam’ and age > 50”

RA

$\sigma_{pName='Adam' \wedge pAge > 50}(Patient)$

TRC

$\{p \mid p \in Patient \wedge p.pName = 'Adam' \wedge p.pAge > 50\}$

DRC

$\{pID, pName, pAge \mid Patient(pID, pName, pAge) \wedge pName = 'Adam' \wedge pAge > 50\}$

SQL

SELECT * FROM Patient

WHERE pName = 'Adam'

AND pAge > 50;

Projection

Find pID of patients whose name is 'Adam' and age > 50

RA

$$\pi_{pID}(\sigma_{pName='Adam' \wedge pAge > 50}(Patient))$$

TRC

$$\{p.pID \mid p \in Patient \wedge p.pName = 'Adam' \wedge p.pAge > 50\}$$

DRC

$$\{pID \mid \exists pName, pAge (Patient(pID, pName, pAge) \wedge pName = 'Adam' \wedge pAge > 50)\}$$

SQL

```
SELECT pID FROM Patient
WHERE pName = 'Adam'
      AND pAge > 50;
```

Join them up

Find the name of patients who have an appointment today

RA

$$\pi_{pName}(\sigma_{date(aDatetime)=today()}(Patient \bowtie Appointment))$$

TRC

$$\{p.pName \mid p \in Patient \wedge a \in Appointment \wedge p.pID = a.pID \wedge date(a.aDatetime) = today()\}$$

DRC

$$\{pName \mid \exists pID, pAge, dID, aDatetime (Patient(pID, pName, pAge) \wedge Appointment(pID, dID, aDatetime) \wedge date(aDatetime) = today())\}$$

SQL

```
SELECT DISTINCT p.pName FROM Patient p
JOIN Appointment a ON p.pID = a.pID
WHERE DATE(a.aDatetime) = CURRENT_DATE;
```

Not Existing

Find name of patients who have no appointment at all

RA

$$\pi_{pName}(Patient) - \pi_{pName}(Patient \bowtie Appointment)$$

TRC

$$\{p.pName \mid p \in Patient \wedge \neg \exists a \in Appointment(a.pID = p.pID)\}$$

DRC

$$\{pName \mid \exists pID, pAge (Patient(pID, pName, pAge) \wedge \neg \exists dID, aDatetime (Appointment(pID, dID, aDatetime)))\}$$

SQL

```
SELECT p.pName FROM Patient p
WHERE NOT EXISTS
(SELECT * FROM Appointment a WHERE a.pID = p.pID)
```

All

Find name of patients who have an appointment with every doctor

RA

$$\pi_{pName}(\text{Patient} \div (\text{Appointment} \div \text{Doctor}))$$

TRC

$$\{p.pName \mid p \in \text{Patient} \wedge \neg \exists d \in \text{Doctor} (\neg \exists a \in \text{Appointment} (a.pID = p.pID \wedge a.dID = d.dID))\}$$

DRC

$$\{p.pName \mid p \in \text{Patient} \wedge \forall d \in \text{Doctor} \exists a \in \text{Appointment} (a.pID = p.pID \wedge a.dID = d.dID)\}$$

SQL

```
SELECT p.pName FROM Patient p
WHERE NOT EXISTS (
    SELECT * FROM Doctor d
    WHERE NOT EXISTS (
        SELECT * FROM Appointment a
        WHERE a.pID = p.pID
        AND a.dID = d.dID
    )
);
```

Schema 1: Employee / Manager (Self-relationship)

Employee(EID, EName, Dept, Salary, ManagerID)

Practice queries

1. Employees with **no manager** ✓
2. Employees who **manage no one** ✓
3. Employees whose **manager works in the same department**
4. Highest-paid employee(s) ! ($\neg \exists$ or $\forall$)
5. Employees who earn **more than their manager** !
6. (Challenge) Managers who manage **all employees in their department**

Schema 2: Student / Course / Registration

Student(SID, SName, Major, Year)

Course(CID, CName, Credits, Dept)

Register(SID, CID, Grade)

Practice queries

1. Students who register **at least one course** ✓
2. Students who register **no course** ✓
3. Students who register “**Database Systems**” ✓
4. Students who get **only A grades** !
5. Students who register **ALL courses in a department** ✓✓ (division)
6. (Challenge) Courses that **all students** register for ! !

Schema 3: Product / Order / Payment

Product(PID, PName, Price)

Order(OID, OrderDate)

OrderItem(OID, PID, Quantity)

Payment(OID, PayDate, Amount)

Practice queries

1. Products **never ordered** 
2. Orders with **no payment** 
3. Products ordered **today**
4. Customers/orders with **multiple products**
5. Orders where **total payment < total order cost** 
6. (Challenge) Products ordered in **every order**   (division)

Schema 1: Employee / Manager (Self-relationship)

Employee(EID, EName, Dept, Salary, ManagerID)

1. Employees with **no manager** ✓

RA : $\sigma_{\text{ManagerID IS NULL}}(\text{Employee})$ SQL : SELECT * FROM Employee
TRC : $\{e | e \in \text{Employee} \wedge e.\text{ManagerID IS NULL}\}$ WHERE ManagerID IS NULL;
DRC : $\{eID, eN, d, s, mID | \text{Employee}(eID, eN, d, s, mID) \wedge mID \text{ IS NULL}\}$

2. Employees who **manage no one** ✓

RA : $\pi_{EID, EName}(\text{Employee}) - \pi_{e.EID, e.EName}(\rho_e(\text{Employee}) \bowtie_{e.EID = m.ManagerID} \rho_m(\text{Employee}))$
TRC : $\{e | e \in \text{Employee} \wedge \neg \exists m \in \text{Employee} (m.ManagerID = e.EID)\}$
DRC : $\{eID, eN, d, s, mID | \text{Employee}(eID, eN, d, s, mID) \wedge \neg \exists mID, mN, md, ms (\text{Employee}(mID, mN, md, ms, eID))\}$
SQL : SELECT * FROM Employee e
WHERE NOT EXIST (SELECT 1 FROM Employee m
WHERE m.ManagerID = e.EID)

3. Employees whose **manager works in the same department**

RA : $\pi_{e.*}(\rho_e(\text{Employee}) \bowtie_{e.ManagerID = m.EID \wedge e.Dept = m.Dept} \rho_m(\text{Employee}))$
TRC : $\{e | e \in \text{Employee} \wedge \exists m \in \text{Employee} (e.ManagerID = m.EID \wedge e.Dept = m.Dept)\}$
DRC : $\{eID, eN, d, s, mID | \text{Employee}(eID, eN, d, s, mID) \wedge \exists m_eN, m_s, m_mID (\text{Employee}(mID, m_eN, d, m_s, m_mID))\}$
SQL : SELECT e.* FROM Employee e JOIN Employee m ON e.ManagerID = m.EID WHERE e.Dept = m.Dept;

4. Highest-paid employee(s) ! ($\neg \exists$ or $\forall$)

RA : $\text{Employee} - \pi_{e1.*}(\rho_{e1}(\text{Employee}) \bowtie_{e1.Salary < e2.Salary} \rho_{e2}(\text{Employee}))$
TRC : $\{e | e \in \text{Employee} \wedge \forall x \in \text{Employee} (e.Salary \geq x.Salary)\}$
DRC : $\{eID, eN, d, s, mID | \text{Employee}(eID, eN, d, s, mID) \wedge \forall xID, xN, xd, xs, xmID (\text{Employee}(xID, xN, xd, xs, xmID) \rightarrow s \geq xs)\}$
SQL : SELECT e.* FROM Employee WHERE Salary $\geq$ ALL (SELECT Salary FROM Employee);

5. Employees who earn **more than their manager** !

RA : $\pi_{e.*}(\rho_e(\text{Employee}) \bowtie_{e.ManagerID = m.EID \wedge e.Salary > m.Salary} \rho_m(\text{Employee}))$
TRC : $\{e | e \in \text{Employee} \wedge \exists m \in \text{Employee} (e.ManagerID = m.EID \wedge e.Salary > m.Salary)\}$
DRC : $\{eID, eN, d, s, mID | \text{Employee}(eID, eN, d, s, mID) \wedge \exists m_eN, md, ms, mmID (\text{Employee}(mID, m_eN, md, ms, mmID) \wedge s > ms)\}$
SQL : SELECT e.* FROM Employee e JOIN Employee m ON e.ManagerID = m.EID WHERE e.Salary > m.Salary;

6. (Challenge) Managers who manage **all employees in their department**

RA : $\pi_{\text{ManagerID}, \text{Dept}}(\text{Employee}) \div \pi_{EID, \text{Dept}}(\text{Employee})$
TRC : $\{m | m \in \text{Employee} \wedge \forall e \in \text{Employee} (e.Dept = m.Dept \rightarrow e.ManagerID = m.EID)\}$
DRC : $\{mID, mN, ms, mmID | \text{Employee}(mID, mN, ms, mmID) \wedge \forall eID, eN, s, m2 (\text{Employee}(eID, eN, md, s, m2) \rightarrow m2 = mID)\}$
SQL : SELECT * FROM Employee m WHERE NOT EXIST (SELECT EID FROM Employee e WHERE e.Dept = m.Dept
EXCEPT SELECT EID FROM Employee WHERE ManagerID = m.EID);

Schema 2: Student / Course / Registration

Student(SID, SName, Major, Year)

Course(CID, CName, Credits, Dept)

Register(SID, CID, Grade)

1. Students who register **at least one course** ✓

RA : $\pi_{SID, SName}(Student \bowtie Register)$

TRC : $\{s | s \in Student \wedge \exists r \in Register (r.SID = s.SID)\}$

DRC : $\{sID, sN, M, Y | Student(sID, sN, M, Y) \wedge \exists cID, G (Register(sID, cID, G))\}$

SQL : SELECT DISTINCT s.* FROM Student s JOIN Register r ON s.SID = r.SID

2. Students who register **no course** ✓

RA : $\pi_{SID}(Student) - \pi_{SID}(Register)$

TRC : $\{s | s \in Student \wedge \neg \exists r \in Register (r.SID = s.SID)\}$

DRC : $\{sID, sN, M, Y | Student(sID, sN, M, Y) \wedge \neg \exists cID, G (Register(sID, cID, G))\}$

SQL : SELECT * FROM Student WHERE SID NOT IN (SELECT SID FROM Register);

3. Students who register **"Database Systems"** ✓

RA : $\pi_{Student.*}(Student \bowtie Register \bowtie \sigma_{CName='Database Systems'}(Course))$

TRC : $\{s | s \in Student \wedge \exists r \in Register, c \in Course (r.SID = s.SID \wedge r.CID = c.CID \wedge c.CName = 'Database Systems')\}$

DRC : $\{sID, sN, M, Y | Student(sID, sN, M, Y) \wedge \exists cID, G, CR, D (Register(sID, cID, G) \wedge c.CName = 'Database Systems')\}$

SQL : SELECT s.* FROM Student s JOIN Register r ON s.SID = r.SID JOIN Course c ON r.CID = c.CID
WHERE c.CName = 'Database Systems';

4. Students who get **only A grades** !

RA : $\pi_{SID}(\sigma_{Grade='A'}(Register)) - \pi_{SID}(\sigma_{Grade \neq 'A'}(Register))$

TRC : $\{s | s \in Student \wedge \exists r \in Register (r.SID = s.SID \wedge r.Grade = 'A') \wedge \forall r2 \in Register (r2.SID = s.SID \rightarrow r2.Grade = 'A')\}$

DRC : $\{sID, sN, M, Y | Student(sID, sN, M, Y) \wedge \exists cID, G (Register(sID, cID, 'A')) \wedge \neg \exists cID2, G2 (Register(sID, cID2, G2) \wedge G2 \neq 'A')\}$

SQL : SELECT s.* FROM Student s WHERE SID IN (SELECT SID FROM Register WHERE Grade = 'A') AND SID NOT IN
(SELECT SID FROM Register WHERE Grade <> 'A');

5. Students who register **ALL courses in a department** ✓✓ (division) → ลมพทเป็น CS

RA : $\pi_{SID, CID}(Register) \div \pi_{CID}(\sigma_{Dept='CS'}(Course))$

TRC : $\{s | s \in Student \wedge \forall c \in Course (c.Dept = 'CS' \rightarrow \exists r \in Register (r.SID = s.SID \wedge r.CID = c.CID))\}$

DRC : $\{sID, sN, M, Y | Student(sID, sN, M, Y) \wedge \forall cID, cN, cR (Course(cID, cN, cR, 'CS') \rightarrow \exists G (Register(sID, cID, G)))\}$

SQL : SELECT s.* FROM Student s WHERE NOT EXISTS (SELECT CID FROM Course WHERE Dept = 'CS' EXCEPT
SELECT CID FROM Register WHERE SID = s.SID);

6. (Challenge) Courses that **all students** register for ! !

RA : $\pi_{CID, SID}(Register) \div \pi_{SID}(Student)$

TRC : $\{c | c \in Course \wedge \forall s \in Student (\exists r \in Register (r.SID = s.SID \wedge r.CID = c.CID))\}$

DRC : $\{cID, cN, cR, D | Course(cID, cN, cR, D) \wedge \forall sID, sN, M, Y (Student(sID, sN, M, Y) \rightarrow \exists G (Register(sID, cID, G)))\}$

SQL : SELECT c.* FROM Course c WHERE NOT EXISTS (SELECT SID FROM Student EXCEPT SELECT SID FROM Register
WHERE CID = c.CID);

Schema 3: Product / Order / Payment

Product(PID, PName, Price)

Order(OID, OrderDate)

OrderItem(OID, PID, Quantity)

Payment(OID, PayDate, Amount)

1. Products **never ordered** ✓

RA : $\pi_{PID}(Product) - \pi_{PID}(OrderItem)$

TRC : $\{p \mid p \in Product \wedge \neg \exists oi \in OrderItem (oi.PID = p.PID)\}$

DRC : $\{pID, pN, pR \mid Product(pID, pN, pR) \wedge \neg \exists oID, Q (OrderItem(oID, pID, Q))\}$

SQL : SELECT * FROM Product WHERE PID NOT IN (SELECT PID FROM OrderItem);

2. Orders with **no payment** ✓

RA : $\pi_{OID}(Order) - \pi_{OID}(Payment)$

TRC : $\{o \mid o \in Order \wedge \neg \exists pay \in Payment (pay.OID = o.OID)\}$

DRC : $\{oID, oD \mid Order(oID, oD) \wedge \neg \exists pID, A (Payment(oID, pID, A))\}$

SQL : SELECT * FROM Order WHERE OID NOT IN (SELECT OID FROM Payment);

3. Products ordered **today**

RA : $\pi_{Product.*}(Product \bowtie OrderItem \bowtie \sigma_{OrderDate = today}(Order))$

TRC : $\{p \mid p \in Product \wedge \exists oi \in OrderItem, o \in Order (oi.PID = p.PID \wedge oi.OID = o.OID \wedge o.OrderDate = today)\}$

DRC : $\{pID, pN, pR \mid Product(pID, pN, pR) \wedge \exists oID, Q (OrderItem(oID, pID, Q) \wedge Order(oID, today))\}$

SQL : SELECT DISTINCT p.* FROM Product p JOIN OrderItem oi ON p.PID = oi.PID JOIN Order o ON oi.OID = o.OID
WHERE o.OrderDate = CURRENT_DATE;

4. Customers/orders with **multiple products**

RA : $\pi_{oi1.OID}(\sigma_{oi1.PID \neq oi2.PID}(\rho_{oi1}(OrderItem) \bowtie_{oi1.OID = oi2.OID} \rho_{oi2}(OrderItem)))$

TRC : $\{o \mid o \in Order \wedge \exists oi1, oi2 \in OrderItem (oi1.OID = o.OID \wedge oi2.OID = o.OID \wedge oi1.PID \neq oi2.PID)\}$

DRC : $\{oID, OD \mid Order(oID, OD) \wedge \exists p1, q1, p2, q2 (OrderItem(oID, p1, q1) \wedge OrderItem(oID, p2, q2) \wedge p1 \neq p2)\}$

SQL : SELECT OID FROM OrderItem GROUP BY OID HAVING COUNT(PID) > 1;

5. Orders where **total payment < total order cost** !

SQL : SELECT t1.OID FROM (SELECT oi.OID, SUM(oi.Quantity * p.Price) AS TotalCost FROM OrderItem oi
JOIN Product p ON oi.PID = p.PID) t1
JOIN (SELECT OID, SUM(Amount) AS TotalPaid FROM Payment GROUP BY OID) t2
WHERE t2.TotalPaid < t1.TotalCost;

6. (Challenge) Products ordered in **every order** ✓✓ (division)

RA : $\pi_{PID,OID}(OrderItem) \div \pi_{OID}(Order)$

TRC : $\{p \mid p \in Product \wedge \forall o \in Order (\exists oi \in OrderItem (oi.PID = p.PID \wedge oi.OID = o.OID))\}$

DRC : $\{pID, pN, pR \mid Product(pID, pN, pR) \wedge \forall oID, OD (Order(oID, OD) \rightarrow \exists q (OrderItem(oID, pID, q)))\}$

SQL : SELECT p.* FROM Product p WHERE NOT EXISTS (SELECT OID FROM Order EXCEPT SELECT OID
FROM OrderItem oi WHERE oi.PID = p.PID);